How a Bloom filter says “maybe”

A Bloom filter stores a set as a handful of bits. It can say “maybe” to a word it never saw, but never “no” to one it did.

Based on Bloom Filters by Example, Bill Mill

1 A set made of bits

Suppose you want to remember which words you have seen without storing the words. A Bloom filter starts with a row of m bits, all 0. To add a word, you run it through k hashes. A hash is a fixed recipe that turns a word into a position in the row, the same position every time for the same word. Each of the k hashes picks one of the m bits, and you set those bits to 1. That is all adding does: the words are not kept, only bits.

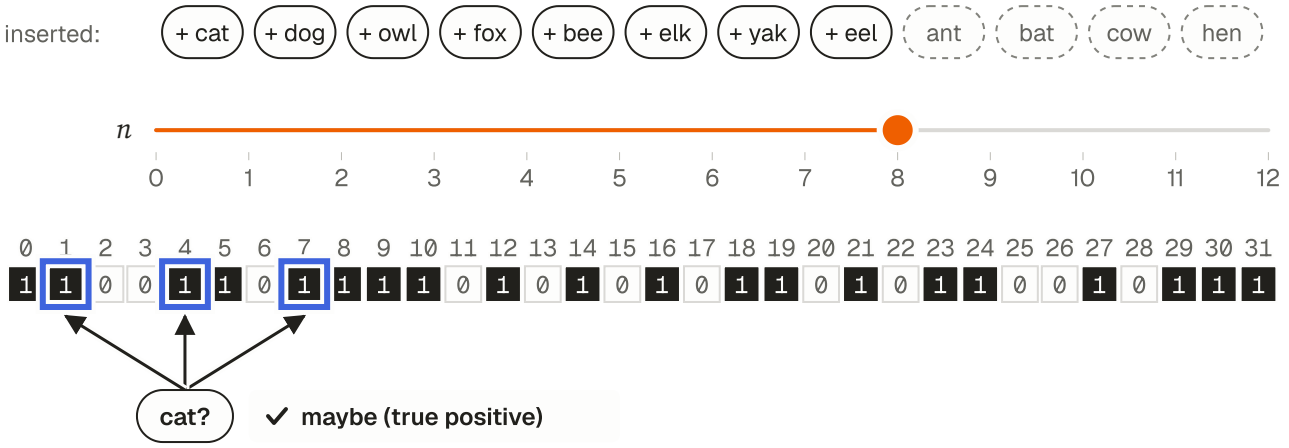
To ask whether a word is in the set, you hash it the same k ways and look at its bits. If any of them is 0, the word was never added: *definitely not*. If all of them are 1, the word was probably added, but you cannot be sure, because other words may have set those bits. The filter's honest answer is *maybe*.

2 Watch it fill up

The figure has 32 bits and a list of twelve animal names. Drag n to add the first n animals, and change k, the number of hashes per word. Then pick a word to ask about. Arrows run from the word to its k bits.

Start with “cat”, which is in the set: all three of its bits are lit, and they always will be. Now ask about “plum”. One of its bits is 0, and that single 0 is proof. Then ask about “cherry”. You never added it, yet all three of its bits are 1: a false positive, a “maybe” that is wrong. At $k = 3$, “cherry” happens to land on exactly the same three bits as “fox”.

Inserting words, asking about words



the filter can't tell this apart from a false positive; the figure knows because it keeps the list

`bits set 20 / 32  predicted FPR 14.7%  observed FP 10 / 50`

Shown at $n = 8$, $k = 3$, ask about “cat”. Interactive version at explorable.geekplux.com/s/bloom-filter#fig-bits

Each word is hashed k times and sets k of the 32 bits. Drag n to insert more animals, change k, and pick a word to ask about. Hatching marks a false “maybe”.

3 Why “no” is always true

Bits only ever go from 0 to 1. Adding a word never clears a bit, so a word you added keeps all of its bits lit for good, and the filter can never answer “definitely not” about it. Every mistake runs one way: a word you never added finds all of its bits already set by others.

Ask about “hen”, the twelfth animal, and drag n up from 8. At $n = 11$ “hen” already says maybe, before you have added it: other animals set its bits. At $n = 12$ it is in the set for real. From the outside the two answers look the same, and that is exactly what “maybe” means. The figure can tell them apart only because it keeps the list; the filter cannot.

4 Predicting the false positives

You can predict how often “maybe” is wrong. A bit stays 0 only if every one of the $k \cdot n$ hashes so far has missed it. Each hash misses a given bit easily, but there are many of them, so the chance that a bit is still 0 shrinks steadily as words go in.

$$(1 - 1/m)^{kn} \approx e^{-kn/m}$$

Each of the $k \cdot n$ hashes misses a given bit with chance $1 - 1/m$, so this is the chance that a bit is still 0 after n words.

A word you never added is a false positive when all k of its bits happen to be 1. So the chance of a false positive is the chance that one bit is 1, multiplied by itself k times. For $m = 32$, $n = 8$ and $k = 3$ that comes to about 14.7%.

$$\text{FPR} \approx (1 - e^{-kn/m})^k$$

The false-positive rate is the chance that one bit is 1, raised to the power k: all k of the word's bits must be 1 at once.

The figure also measures. It asks about 50 fruits and vegetables it never added and counts how many come back maybe. At the defaults that is 10 of 50, or 20%, a little above the prediction. The prediction describes an average filter. This one happens to have 20 of its 32 bits set rather than the expected 17, and with 20 of 32 bits set, all three of a word's bits are 1 about 24% of the time. A filter this small is noisy; with more bits, the measured rate settles onto the prediction.

$$(20/32)^3 \approx 24\%$$

Each of the three bits lands on a set bit with chance 20/32, so all three do with that chance cubed.

5 Choosing k

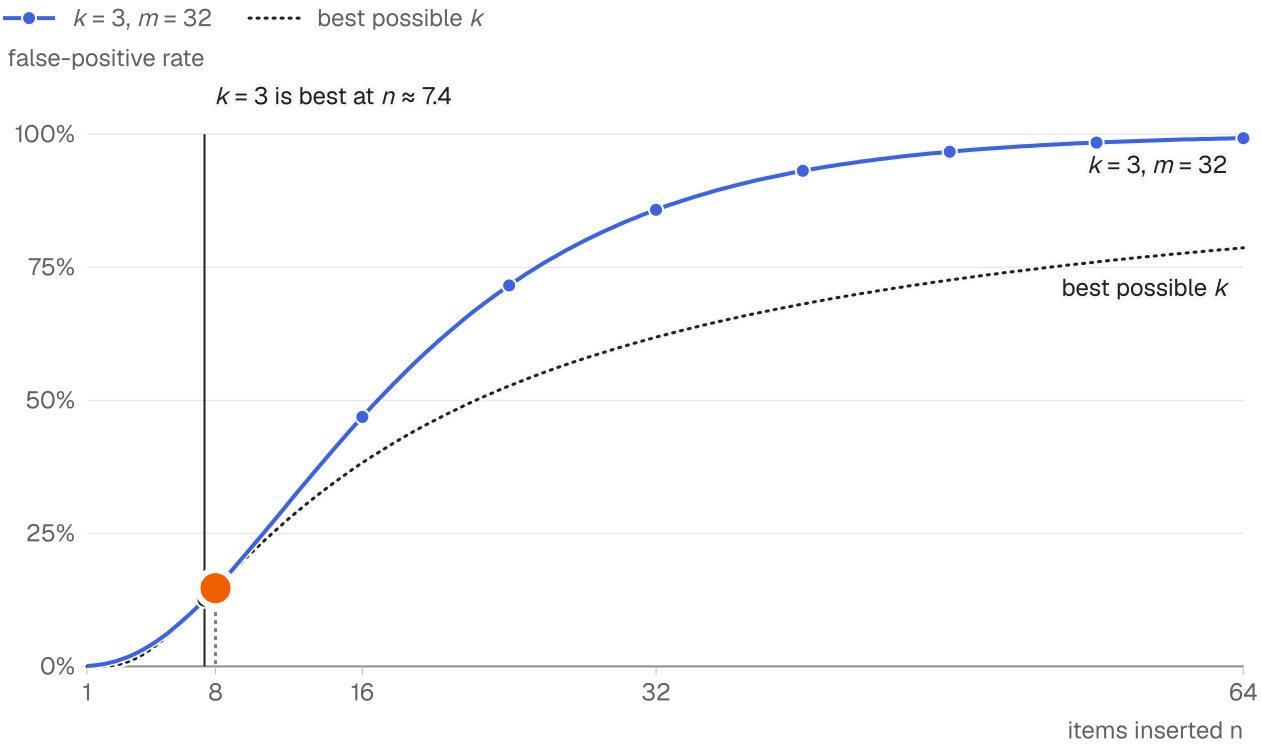
More hashes cut both ways. Each extra hash is one more bit a word you never added has to match, which helps, but it also fills the bits faster, which hurts. In the first figure at $n = 8$, the predicted rate is 22% at $k = 1$, 14.7% at $k = 3$ and 22% again at $k = 6$. Somewhere in between is a best k.

The best k is about 0.69 times the number of bits per word, m divided by n. That is the point where about half the bits are set, and there each extra bit per word cuts the false-positive rate to about 62% of what it was. Drag along the curve and watch it touch the dotted curve at the marker.

$$k = (m/n) \cdot \ln 2, \text{ where } \text{FPR} = (1/2)^k \approx 0.6185^{m/n}$$

Setting the slope of the rate to zero gives this k. Half the bits are then set, so each of a word's k bits is 1 half the time.

False positives as the filter fills



`items inserted n` **8** `false-positive rate` **14.7%**

Shown at $n = 8$, $k = 3$, $m = 32$. Interactive version at explorable.geekplux.com/s/bloom-filter#fig-fpr-curve

The curve is the predicted rate of false positives for your m and k as n grows. The dotted curve is the best any number of hashes could do; the marker shows where your k is the best choice.

Takeaway

A Bloom filter never says no to a word it holds, because bits only turn on. Its one error is a false “maybe”, and how often it happens follows from the number of bits, words and hashes. It is rarest when about half the bits are set.

Read the original post